Wine Classification - Complete Project Workflow Guide

Project goal

Build a reproducible multi-class classifier that predicts one of three wine cultivars from 13 chemical measurements. This is a learning project for practicing data science, statistics, and model validation.

Source and data collection

Original source: UCI Machine Learning Repository, Wine dataset, https://archive.ics.uci.edu/dataset/109/wine. The same task also appears on Kaggle: https://www.kaggle.com/c/wine-m/data, but that Kaggle competition contains modified files. This project uses the original 178-row UCI version distributed with scikit-learn, not the Kaggle competition files.

Stage 1: Run 01_Data_Collection.ipynb. Review the feature list, first records, target categories, dataset size and save wine_raw.csv.

Data preprocessing

Stage 2: Run 02_Data_Preprocessing.ipynb. Check data types, nulls, exact duplicates, invalid target labels and potential IQR outliers. Save wine_clean.csv. The original data contains no missing values; do not claim to have fixed missing data.

The machine-learning pipeline applies median imputation and standard scaling within cross-validation folds. This avoids using test-set statistics during training.

Univariate and bivariate analysis

Stage 3: Read EDA_Questions_Only.pdf first. Open 03_Univariate_Bivariate_Answers.ipynb and run each numbered section for its matching answer and plot.

Univariate: class counts, central tendency, spread, alcohol histogram, outlier flags. Bivariate: correlations, scatterplots, class-wise boxplots, heatmap and exploratory ANOVA.

Correlations show association, not causation. Group comparisons are exploratory; multiple-testing correction is not applied.

Feature selection and model creation

Stage 4: Split data into stratified training (80%) and test (20%) sets using random_state=42. Build a Pipeline with imputation, scaling, SelectKBest and a classifier.

Use 5-fold stratified GridSearchCV with macro F1 to compare Logistic Regression, Support Vector Machine and Random Forest and feature counts 5, 8, and 13. Hyperparameter selection uses only training folds.

Save the winning fitted pipeline and metadata to trained_model.joblib. Do not inspect test performance while selecting hyperparameters.

Final model and evaluation

Stage 5: Run 05_Final_Model.ipynb. Load the trained pipeline, reconstruct the same held-out split and evaluate once. Record accuracy, macro F1, precision, recall, per-class F1 and the confusion matrix.

Save test_predictions.csv. The notebook predicts one example with the saved model. Model outputs are educational, not a validated production system.

How to present the assignment

Start with the problem statement and source. Show the dataset shape and class distribution. Explain preprocessing decisions and why data leakage matters. Present two univariate and two bivariate plots. Explain how GridSearchCV selects features and hyperparameters. Finish with the actual held-out metrics and limitations.

A simple closing line: "I followed the full workflow from data collection to model evaluation. I used cross-validation for model selection and kept the test set separate to get a fair final result."

Running order and requirements

Install: pip install pandas numpy matplotlib seaborn scipy scikit-learn joblib jupyter

Open the project root in Jupyter and run the notebooks in order 01, 02, 03, 04, 05. Stage 4 may take longer because it trains multiple model configurations.

Keep the folder structure intact. The PDF in stage 3 contains questions only; the matching answers are executable notebook cells.

Limitations

Only 178 rows and three relatively separable classes. The model may not generalize to new laboratories, regions or measurement protocols. GridSearchCV is not a substitute for external validation.